

GRAPH REPRESENTATION (Adjacency List)

```
vector<vector<int>>> adj(n); // unweighted
adj[u].push_back(v); adj[v].push_back(u); // undirected edge u-v
adj[u].push_back(v); // directed edge u->v only
vector<vector<pair<int,int>>>> wadj(n); // weighted: {neighbor, weight}
wadj[u].push_back({v, w});
```

DFS (Depth First Search)

Use: exploring as deep as possible before backtracking connectivity, cycles, paths.

```
vector<bool> visited(n, false);
void dfs(int u) {
    visited[u] = true;
    // process u here
    for (int v : adj[u]) if (!visited[v]) dfs(v);
}
// iterative version (avoids stack overflow on deep graphs)
void dfsIter(int start) {
    stack<int> st; st.push(start);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (visited[u]) continue;
        visited[u] = true; // process u here
        for (int v : adj[u]) if (!visited[v]) st.push(v);
    }
}
```

BFS (Breadth First Search)

Use when: exploring level-by-level – shortest path in unweighted graphs.

```
vector<bool> visited(n, false); vector<int> dist(n, -1);
void bfs(int start) {
    queue<int> q; q.push(start);
    visited[start] = true; dist[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) if (!visited[v]) {
            visited[v] = true; dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
} // dist[v] after bfs = shortest #edges from start to v (unweighted only)
===== WEIGHTED EDGE LIST INPUT (common contest input format) =====
vector<vector<pair<int,int>>>> wadj(n);
for (int i=0;i<m;i++) {
    int u,v,w; cin>>u>>v>>w; u--; v--; // convert 1-indexed to 0-indexed
    wadj[u].push_back({v,w}); wadj[v].push_back({u,w});
}
===== BFS SHORTEST PATH RECONSTRUCTION =====
vector<int> parent(n, -1);
// inside bfs, when visiting v from u: parent[v] = u;
vector<int> path;
for (int v=target; v!=-1; v=parent[v]) path.push_back(v);
reverse(path.begin(), path.end());
// U R D L
int dx[] = {-1,0,1,0};
int dy[] = {0,1,0,-1};
```